ASSIGNMENT 1 - TASK 11: Implement, Test and Defend the Design

Name: K.V.N.D.P.SAI RUPESH | Register Number: 1972260102 | Bloom's Level: K5 - Evaluate | Marks: 5

Source code: ParkingSystem.java (same folder). Output screenshots: see the screenshot files in this folder. All tests below were executed and the actual results are copied from the program output.

1. Test table (facility with floor 0 = 2 slots, floor 1 = 1 slot)

No.	Test	Category	Expected	Actual	Result
1	Two-wheeler enters	Normal	[0, 0]	[0, 0]	PASS
2	Car enters	Normal	[0, 1]	[0, 1]	PASS
3	EV takes the last free slot	Boundary	[1, 0]	[1, 0]	PASS
4	Facility full, new car arrives	Boundary	null	null	PASS
5	Duplicate registration TN01	Invalid	rejected	rejected	PASS
6	Unknown vehicle type TRUCK	Invalid	rejected	rejected	PASS
7	Invalid slot number release(5, 0)	Invalid	false	false	PASS
8	Zero-minute stay	Boundary	0.0	0.0	PASS
9	Exactly 60 minutes, car	Boundary	30.0	30.0	PASS
10	61 minutes, car	Boundary	60.0	60.0	PASS
11	EV, 60 min + 5 kWh	Normal	90.0	90.0	PASS
12	Day total (hand calculated)	Scenario	190.0	190.0	PASS

Summary printed by the program: Passed: 12 Failed: 0

2. Hand calculation of the day's total collection

Tariff: two-wheeler Rs 10 per started hour; car Rs 30 per started hour; EV parking Rs 30 per started hour plus Rs 12 per kWh charged (charging is separate); valid monthly pass parks free.

Vehicle	Stay	Calculation	Collection
TN11 two-wheeler	45 min	1 started hour x 10	Rs 10
TN12 car	130 min	3 started hours x 30	Rs 90
TN13 EV	60 min, 5 kWh	(1 x 30) + (5 x 12)	Rs 90
TN14 car with monthly pass	200 min	Parking waived	Rs 0
Day total		10 + 90 + 90 + 0	Rs 190

The program printed 190.0, which matches the hand calculation. The fee uses integer arithmetic, hours = (minutes + 59) / 60, so 130 minutes gives 3 hours (plain 130 / 60 would give 2 and undercharge).

3. Evaluation of the design

- **Encapsulation:** fields are private; invalid values (negative minutes, negative kWh, empty registration number) are rejected inside the class.

- **Inheritance:** Car and ElectricCar reuse the fee logic of Vehicle; ElectricCar adds only the charging fee.

- **Polymorphism:** calculateFee() calls each object's own ratePerHour(), so no if-else chain on vehicle type is needed.

- **Abstraction:** Vehicle is an abstract class and Chargeable is an interface.

- **Limitation:** adding a new vehicle type needs a new subclass and also an edit to the createVehicle() switch, so one existing class must change. Also, all data is in memory and is lost when the program stops.

4. Alternative: array, ArrayList or HashMap for active tickets

Structure	Find ticket on exit	Space for n vehicles	Note
Array / ArrayList of tickets	O(n), scan each ticket	O(n)	Duplicate check also O(n)
HashMap keyed by registration number (used in my program)	O(1) on average	O(n), slightly more per entry	Duplicate check is O(1)

Exit processing with a HashMap: get the ticket by registration number, free its slot, remove it from the map, then calculate the fee. With a list, the same step first needs a loop to find the ticket. For a campus facility the HashMap is the better choice.